International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Lab 7 – Test on OOP Concepts (Set B)

Date: 11.09.2026 **Semester:** B.Tech 3rd Semester **Group:** 1

Instructions

- Write all programs in **C++**.
- Use the OOP concepts specified in each question appropriately.
- Use **pointers** and dynamic memory allocation wherever required.
- Properly release dynamically allocated memory using `delete` or `delete[]`.
- Display the output in a clear and readable format.

1. **Parking Slot Tracker**
   A parking building contains a number of floors entered at runtime. Create a class `ParkingFloor` containing the floor number, number of slots, and a dynamically allocated array representing slot occupancy.

   Implement overloaded `reserve()` functions: one reserves a particular slot, while another reserves a consecutive group of slots starting from a given position. Create the floors dynamically and correctly release all allocated memory.

2. **Library Fine Calculator**
   Create a class `Borrower` containing borrower ID, overdue days, and a private fine amount.

   Implement overloaded `calculateFine()` functions: one calculates the fine using only the overdue days, while another uses the overdue days and a special-rate value.

   Write a friend function that compares the private fine amounts of two borrowers and identifies the borrower with the higher fine. Create the borrowers dynamically.

3. **Weather Sensor Grid**
   A weather station contains n sensors, where n is entered at runtime. Create a class `SensorGrid` containing a dynamically allocated array of temperature readings.

   Provide functions to enter readings, display readings, and replace a reading at a specified position. Write a friend function that compares two `SensorGrid` objects and determines which has the greater average temperature. The two grids may contain different numbers of readings.

4. **Game Inventory Controller**
   Create a class `Inventory` containing player ID, number of items, and a dynamically allocated array of item quantities.

   Create another class `GameController`. The controller must be able to inspect and modify the private inventory information, including changing the quantity of a particular item, without making the inventory data public.

   Demonstrate the appropriate OOP mechanism. Create an `Inventory` object dynamically and release the memory correctly.

5. **Taxi Fare Manager**
   Create a class `Trip` containing trip ID, distance travelled, and base fare.

   Implement overloaded `fare()` functions:

   - `fare()` calculates the normal fare.
   - `fare(int waitingTime)` adds a waiting charge.
   - `fare(int waitingTime, int discount)` adds the waiting charge and then applies the discount.

   Create two trips dynamically. Write a friend function that compares the final fares of the two trips and reports which trip is cheaper. The comparison must not access the fare through a public data member.